

Laboratory Report
Course: Algorithm Analysis and Design Lab
Experiment No: 2
Experiment Title: All-Pairs Shortest Path Using Recursion and Dynamic Programming

Submitted by: A. S. M. MORSHED
Student ID: MUH2425001M

1.Objectives:

The main objectives of this experiment are:

- To learn the basic concept of Dynamic Programming.
- To understand how recursive shortest path calculation works.
- To identify overlapping sub problems in recursive solutions.
- To reduce repeated calculations using Dynamic Programming.
- To compare the performance of recursive and DP approaches.

2.Theory:

The All-Pairs Shortest Path problem is used to determine the minimum distance between every pair of vertices in a weighted graph.

In this experiment, both recursive and Dynamic Programming methods are used to solve the problem.

Recursive Method:

The recursive method calculates the shortest path by breaking the problem into smaller sub problems and solving them repeatedly.

Recursive relation:

$$\text{Shortest}(i,j,k) = \min(\text{Shortest}(i,j,k-1), \text{Shortest}(i,k,k-1) + \text{Shortest}(k,j,k-1))$$

Because the same sub problems are solved many times, the recursive method becomes slower for larger graphs.

Dynamic Programming Method:

Dynamic Programming stores previously calculated shortest distances in a matrix so that the same calculations do not need to be repeated again.

This improves the overall efficiency of the algorithm.

The Floyd-Warshall Algorithm is used as the Dynamic Programming solution for finding shortest paths between all pairs of vertices.

3. Recursive Algorithms :

Algorithm RecursiveShortest(i, j, k)

```
if(k < 0)
return graph[i][j]
```

```
withoutK = RecursiveShortest(i, j, k - 1)
```

```
withK = RecursiveShortest(i, k, k - 1) + RecursiveShortest(k, j, k - 1)
```

```
return minimum(withoutK, withK)
```

In this recursive approach, the algorithm checks whether using the intermediate vertex k produces a shorter path between two vertices. The same subproblems are solved repeatedly, which increases execution time.

4. Dynamic Programming Algorithm :

Algorithm FloydWarshall()

```
for(k = 0; k < n; k++){
for(i = 0; i < n; i++){
for(j = 0; j < n; j++){

if(dist[i][k] + dist[k][j] < dist[i][j]){
dist[i][j] = dist[i][k] + dist[k][j];
}
}
}
}
```

In the Dynamic Programming approach, previously calculated shortest distances are stored in a matrix. This reduces repeated calculations and improves the overall performance of the algorithm.

5.Source Code :

```
#include <stdio.h>

#define INF 99999
#define MAX 10

int graph[MAX][MAX];
int n;

int RecursiveShortest(int i, int j, int k)
{
    if(k < 0)
    {
        return graph[i][j];
    }

    int withoutK =
    RecursiveShortest(i, j, k - 1);

    int left =
    RecursiveShortest(i, k, k - 1);

    int right =
    RecursiveShortest(k, j, k - 1);

    int withK;

    if(left == INF || right == INF)
    {
        withK = INF;
    }
    else
    {
        withK = left + right;
    }
}
```

```

        if(withoutK < withK)
        {
            return withoutK;
        }
        else
        {
            return withK;
        }
    }
}

int main()
{
    int dist[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);

            dist[i][j] = graph[i][j];
        }
    }

    // Floyd-Warshall Algorithm
    for(int k = 0; k < n; k++)
    {
        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if(dist[i][k] != INF &&

```

```

        dist[k][j] != INF &&
        dist[i][k] + dist[k][j] < dist[i][j])
    {
        dist[i][j] = dist[i][k] + dist[k][j];
    }
}

printf("\nShortest Path Matrix:\n");

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
    {
        if(dist[i][j] == INF)
        {
            printf("INF ");
        }
        else
        {
            printf("%3d ", dist[i][j]);
        }
    }

    printf("\n");
}

printf("\nRecursive shortest path from 0 to 3 = %d\n",
        RecursiveShortest(0, 3, n - 1));

return 0;
}

```

6. Sample Input / Output

Input:

```
Enter number of vertices: 4
Enter adjacency matrix:

0 3 99999 7
8 0 2 99999
5 99999 0 1
2 99999 99999 0
```

Output:

```
Shortest Path Matrix:
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0

Recursive shortest path from 0 to 3 = 6
```

7.Complexity Analysis:

Method	Time Complexity	Space Complexity
Recursive Approach	Exponential	$O(n)$
Dynamic Programming	$O(n^3)$	$O(n^2)$

8. Comparison Between Recursive and DP

Recursive Method	Dynamic Programming
Repeated calculations occur	Stores previous results
Slower execution	Faster execution
Less efficient	More efficient
Higher time complexity	Lower time complexity

9. Conclusion:

In this experiment, the shortest paths between all pairs of vertices were found using both recursive and Dynamic Programming approaches.

The recursive method solved the same sub problems repeatedly, which increased the execution time. On the other hand, the Dynamic Programming approach stored previously calculated values and reduced repeated calculations.

After comparing both methods, it was observed that the Dynamic Programming approach works faster and gives better performance for larger graphs by using the Floyd-Warshall Algorithm.